```
What is OOP (Object-Oriented Programming)?

OOP = Object-Oriented Programming (OOP) is a programming approach where we
use classes and objects to design programs based on real-world concepts.

Instead of only using variables and functions,
we create objects (things) with data + actions.
```

Class & Object in Python

Class is a blueprint for creating objects.

object is a instance of a class.

In [5]:
```
#creating class

class Student:   # class names starts with cap
    name = "Sheela"

#creating object (instance)

s1 = Student() #created object -instance
print(s1.name)

s2 = Student()
print(s2.name)
```

```
Sheela
Sheela
```

In [9]:
```
class Car:
    color="blue"
    brand="mercedes"

car1=Car()

print(car1.color)
print(car1.brand)
```

```
blue
mercedes
```

```
init Function
Constructor

All classes have a function called __init__(), which is always executed
when the class is being initiated.

#creating class

class Student:
```

```
    def __init__(self, fullname):
        self.name = fullname

#creating object

s1 = Student("Sheela")
print(s1.name)

The self parameter is a reference to the current instance of the class, and
is used to access variables that belongs to the class.
```

In [9]:
```
class Student:
    def __init__(self, fullname, marks): #self para can be anything , abc,cd .
        self.name = fullname
        self.marks=marks

#creating object

s1 = Student("Sheela kunwar", 77)
print(s1.name, s1.marks)

s2 = Student("seema kunwar", 99)
print(s2.name, s2.marks)
```

```
Sheela kunwar 77
seema kunwar 99
```

In [ ]:
```
Method
Methods are the functions that belongs to objects.
```

In [3]:
```
class Student:
    def __init__(self, name):
        self.name = name

    def greet(self):   # method
        print("Hello, my name is", self.name)


s1 = Student("Sheela")
s1.greet()
```

```
Hello, my name is Sheela
```

```
greet() - method
self.name - gets value from object
s1.greet() - calls the method
```

In [5]:
```
class Calculator:
    def add(self, a, b):
        print("Sum:", a + b)

c1 = Calculator()
c1.add(1, 3)
```

```
Sum: 4
```

"**init** is a constructor used to initialize object values, and self refers to the current object to store and access its data."

What is Abstraction?

=> Abstraction means:

Show only important details Hide internal implementation

Example:

ATM → you withdraw money You don't see internal banking logic

Why we protect abstraction?

Prevent wrong changes Keep data secure Maintain control over logic Avoid bugs

In [15]:
```
class Student:
    def __init__(self, marks):
        self.marks = marks

s1 = Student(80) #Anyone can change marks, No validation
s1.marks = -40
print(s1.marks)
```

```
-40
```

Double Underscore __variable =>Meaning Makes variable private Python changes its name internally

In [18]:
```
#with protection
class Student:
    def __init__(self, marks):
        self.__marks = marks   # private variable

    def set_marks(self, value):
        if value >= 0:
            self.__marks = value
        else:
            print("Invalid marks")

    def get_marks(self):
        return self.__marks

s1 = Student(80)
s1.set_marks(-40)   # not allowed
print(s1.get_marks())
```

```
Invalid marks
80
```

__marks (double underscore)

Makes variable private Cannot access directly:

Single Underscore _variable => Meaning “Internal use only” (just a convention) Not truly private

In [21]:
```
class Student:
    def __init__(self):
        self._marks = 80

s1 = Student()
print(s1._marks)
```

```
80
```

Key Point You can access it But you should NOT (developer rule)

“Please don’t touch this"

What is **dict**?

dict is a built-in attribute in Python It stores all variables (attributes) of an object or class

In [26]:
```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s1 = Student("Sheela", 77)
s2=Student("seema",88)

print(s1.__dict__)
print(s2.__dict__)
```

```
{'name': 'Sheela', 'marks': 77}
{'name': 'seema', 'marks': 88}
```

In [27]:
```
class Student:
    school = "ABC School"

print(Student.__dict__)
```

```
{'__module__': '__main__', 'school': 'ABC School', '__dict__': <attribute '_
_dict__' of 'Student' objects>, '__weakref__': <attribute '__weakref__' of
'Student' objects>, '__doc__': None}
```

In [28]:
```
class Student:
    def __init__(self):
        self.__marks = 90

s1 = Student()
print(s1.__dict__)
```

```
{'_Student__marks': 90}
```

```
Object = person
__dict__ = ID card (all details stored)
```

In [6]:
```
class Student:
    college_name = "ABC College"

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def welcome(self):
        print("welcome student,", self.name)

    def get_marks(self):
        return self.marks


s1 = Student("karan", 97)
s1.welcome()
print(s1.get_marks())
```

```
welcome student, karan
97
```

```
 What is Inheritance?

=>Inheritance means:

One class uses properties and methods of another class
```

```
Real-Life Example
Parent → Child
Child gets features from parent

Example:

Parent: Animal → eat()
Child: Dog → can use eat()
```

In [ ]:
```
#Basic Syntax

class Parent:
    # properties / methods

class Child(Parent):
    # inherits Parent
```

In [31]:
```
class Animal:
    def eat(self):
        print("eating")

class Dog(Animal):
    def bark(self):
        print("Barking")
d= Dog()
d.eat()
d.bark()
```

```
eating
Barking
```

```
Dog uses both:

its own method
parent method
```

```
What is Method Overriding?

=> When a child class changes the method of parent class

=> Same method name, but different implementation
```

In [32]:
```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self):   # overriding
        print("Dog barks")

d = Dog()
d.sound()
```

```
Dog barks
```

In [1]:
```
class Parent:
    def show(self):
        print("Parent method")

class Child(Parent):
    def show(self):
        print("Child method")

c = Child()
c.show()
```

```
Child method
```

In [34]:
```
class Account:
    def interest(self):
        print("General interest")

class Savings(Account):
    def interest(self):
        print("Savings interest is 5%")

a = Savings()
a.interest()
```

```
Savings interest is 5%
```